FLASHED

Documento técnico: requisitos, requerimientos y arquitectura

Producto	Juego arcade 2D para navegador
Versión	0.1.0 · 10 de octubre de 2026
Tecnología	Phaser 3 · TypeScript · Vite · Web Audio
Repositorio	github.com/jpaquezph/flashed (privado, rama main)

1. Resumen

FLASHED es un juego arcade de un solo jugador que corre completo en el navegador, sin servidor ni cuentas. El jugador es un fotógrafo de eventos que tiene que cumplir los objetivos de cada fiesta mientras esquiva los láseres del DJ, que le queman el sensor de la cámara. La campaña tiene seis eventos y una pelea final contra el DJ.

Todo el contenido se genera en el momento de cargar: los gráficos son SVG armados por código y el sonido se sintetiza con Web Audio. El paquete publicado es un único archivo JavaScript más una página HTML; no hay imágenes, fuentes ni archivos de audio que descargar.

2. Requisitos

2.1 Para jugar

Elemento	Requisito
Navegador	Uno actual con ES2020, WebGL (o Canvas 2D con `?canvas`), Web Audio, Pointer Events y SVG como imagen: Chrome, Edge, Firefox o Safari recientes.
Conexión	Solo para la carga inicial (≈1,67 MB, ≈394 KB comprimido con gzip). Después no hace pedidos de red.
Entrada	Teclado, pantalla táctil, mando estándar o control remoto con flechas, Enter y 0.
Pantalla	Cualquier tamaño; el juego conserva la proporción 16:9. En teléfonos y tabletas se juega en horizontal.
Almacenamiento	Opcional: `localStorage` para recordar idioma y sonido. Sin él, el juego funciona igual.

2.2 Para desarrollar y publicar

Elemento	Requisito
Node.js y npm	Node 20 o superior (desarrollado con Node 24).
Dependencias	`phaser ^3.90` en ejecución; `typescript ^5.6` y `vite ^6` para compilar.
Comandos	`npm install` · `npm run dev` (desarrollo) · `npm run build` (verifica tipos y genera `dist/`) · `npm run preview` (sirve `dist/`).
Hosting	Cualquier hosting de archivos estáticos. La ruta base es relativa (`base: './'`), así que funciona en una subcarpeta. Para pantalla completa y orientación en teléfonos conviene HTTPS.

3. Requerimientos funcionales

ID	Requerimiento
RF-01	**Campaña.** Siete escenarios encadenados: Noche de alabanza, Cumpleaños infantil, Cumpleaños de 15, Boda, Discoteca, Recital y Contra el DJ. Al ganar uno se pasa al siguiente.
RF-02	**Movimiento.** El fotógrafo se mueve en ocho direcciones sobre el piso, choca con muebles y no atraviesa a los invitados.
RF-03	**Fotografía.** Cada disparo evalúa quién quedó dentro del encuadre que se ve en el piso. La foto vale según cantidad de personas, protagonistas y repetición; hay una espera corta entre fotos.
RF-04	**Objetivos obligatorios.** Cada evento tiene un mínimo de fotos válidas y tres objetivos (momentos del evento, personas puntuales, foto grupal). Para salir hay que cumplir todo.
RF-05	**Momentos.** Los momentos del evento (la torta, el vals, el solo) ocurren a los pocos segundos de empezar y, si su objetivo queda pendiente, se repiten.
RF-06	**Tarjeta de memoria.** 10 disparos por evento (30 en la pelea final). Si se llena con objetivos pendientes, se pierde, con una animación propia.
RF-07	**Láseres.** Barridos horizontales, verticales, diagonales y combinados, con aviso previo y un corredor seguro. Cada impacto quema una parte del sensor; sin sensor se pierde.
RF-08	**Flash al iluminador.** En los eventos comunes, fotografiar a quien maneja las luces con los láseres prendidos los apaga un momento, al costo de un disparo.
RF-09	**Salida.** Con todo cumplido, la puerta se ilumina y se sale con la tecla de interactuar. Si se acaba el tiempo antes, se pierde.
RF-10	**Dificultad creciente.** De un evento al siguiente los láseres descansan menos y barren más rápido, hay menos tiempo y más invitados en movimiento.
RF-11	**Jefe final.** Pelea con dos barras de vida. Las fotos le sacan vida al DJ (el doble de cerca). Tres fases: en la segunda se mueve detrás de la cabina; en la tercera baja a la pista y persigue al jugador para pegarle.
RF-12	**Accesorios.** Al ganar un evento se elige uno entre tres al azar, de siete posibles, que dura toda la partida.
RF-13	**Cinemáticas.** Una inicial antes del primer evento y una final después del jefe, con globos de diálogo, voz en código morse y opción de saltarlas.
RF-14	**Menú y selección de evento.** Menú con controles, idioma, efectos y música; pantalla para elegir cualquier evento.
RF-15	**Interfaz de partida.** Evento y fase, contador de fotos, tarjeta, sensor, tiempo, objetivos siempre visibles, avisos y pausa.
RF-16	**Resultados.** Pantalla de victoria con puntaje y bonus, o de derrota con el motivo y opción de reintentar.
RF-17	**Idiomas.** Español e inglés, intercambiables desde el menú.
RF-18	**Sonido.** Una pista musical de género distinto por escenario, música de victoria y de derrota, y efectos. Música y efectos se pueden apagar por separado desde el menú, la partida y la pausa.
RF-19	**Teclado y mouse.** Todo el juego se maneja con teclado; los botones del menú también responden al mouse.
RF-20	**Pantalla táctil.** Palanca virtual, botones de foto, interactuar y pausa, y todas las pantallas responden al toque. En vertical se pide girar el dispositivo.
RF-21	**Mando y control remoto.** Mando estándar en todas las pantallas; con control remoto, las flechas mueven, Enter dispara y 0 interactúa.

4. Requerimientos no funcionales

ID	Requerimiento
RNF-01	**Sin backend.** No hay servidor, base de datos, cuentas ni telemetría. El juego no envía datos.
RNF-02	**Sin recursos externos.** Gráficos y sonido se generan en el navegador; el paquete es un solo archivo JavaScript.
RNF-03	**Adaptación de pantalla.** Resolución lógica fija de 480 × 270, escalada y centrada conservando 16:9. Se reajusta sola al girar el dispositivo o cambiar el tamaño de la ventana.
RNF-04	**Nitidez y memoria.** El arte vectorial se rasteriza a 4× en computadoras y tabletas (lienzo de 1920 × 1080) y a 3× en teléfonos (1440 × 810).
RNF-05	**Uso de memoria.** Los personajes de cada evento se generan al entrar y se descartan al pasar al siguiente.
RNF-06	**Configurable por datos.** Cada escenario es un objeto de configuración; el balance general vive en un archivo de constantes. Agregar un evento no requiere tocar la lógica.
RNF-07	**Calidad de código.** TypeScript en modo estricto, sin variables ni parámetros sin usar. La compilación falla si la verificación de tipos falla.
RNF-08	**Tolerancia.** Sin `localStorage` o sin audio el juego sigue funcionando. Si la pestaña pasa a segundo plano, la música se retoma sin acumular notas.
RNF-09	**Variedad visual.** En una misma escena ningún personaje genérico aparece más de dos veces.
RNF-10	**Legibilidad.** Los peligros siempre se avisan antes de activarse y el jugador se distingue con una marca propia.

5. Arquitectura

El código está en `src/` (unas 6.900 líneas de TypeScript). Se organiza en capas: las **escenas** deciden qué se muestra, los **sistemas de juego** contienen las reglas, la **configuración** describe cada evento, y los módulos de **arte**, **audio**, **textos** y **dispositivo** dan servicio a todo lo demás.

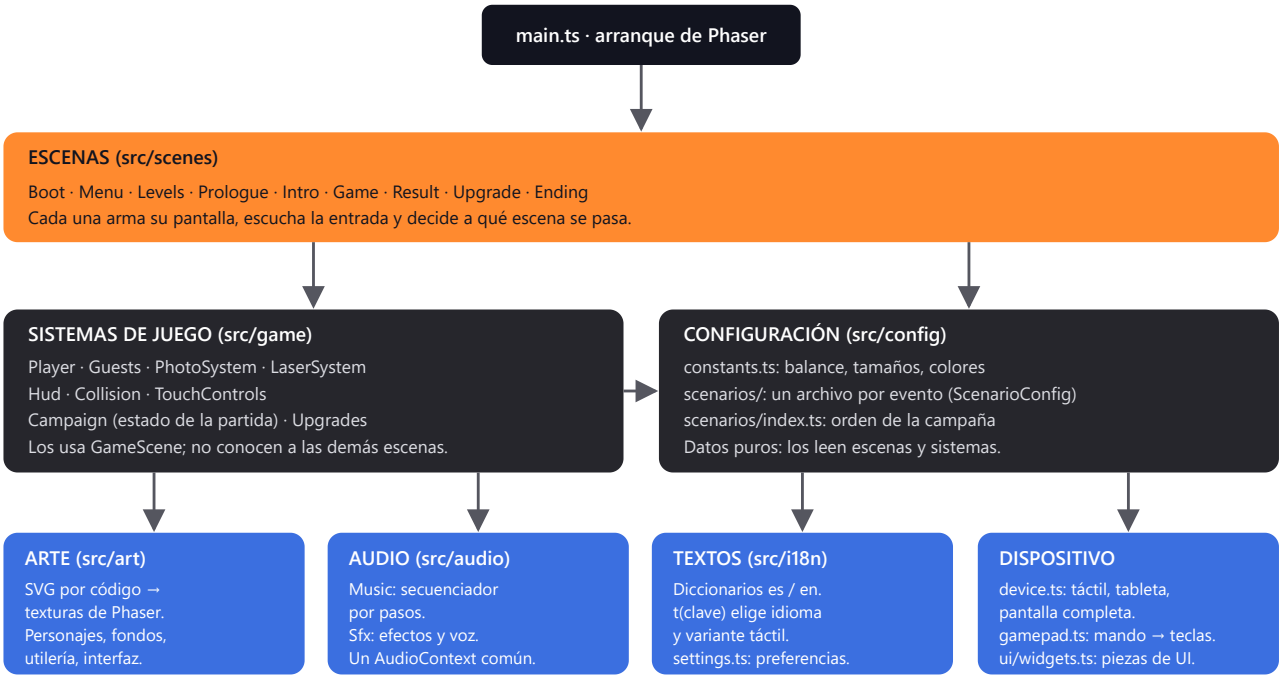


Figura 1. Capas del proyecto. Las flechas indican quién usa a quién.

5.1 Recorrido de pantallas

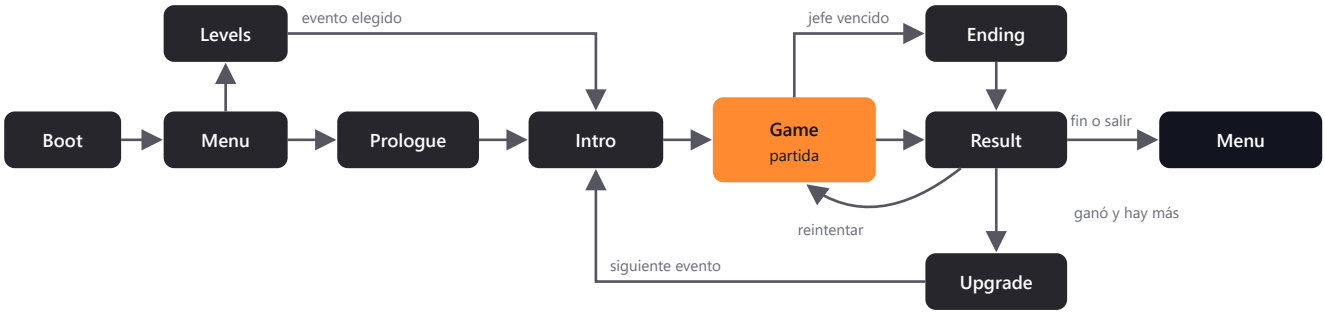


Figura 2. Flujo entre escenas. La cinemática inicial (Prologue) solo aparece antes del primer evento.

5.2 Cómo se conecta una partida

`GameScene` es el coordinador. Al crearse lee la configuración del evento en curso (`Campaign.currentScenario()`) y arma con ella los sistemas. En cada cuadro:

- Junta la entrada de **teclado**, **TouchControls** y, traducida a teclas, del **mando**.
- Avanza los **momentos** del evento y les indica a los actores qué hacer a través de **GuestManager**.
- Mueve al **Player** contra los obstáculos (**Collision**) y a los invitados.
- Actualiza el **LaserSystem**; si un haz toca al jugador, descuenta sensor en **Campaign**.
- Al disparar, pide a **PhotoSystem** el encuadre y el puntaje, y marca objetivos cumplidos.
- Le pasa el estado completo al **Hud**, que solo dibuja.

El estado que sobrevive entre eventos (índice, puntaje, accesorios, sensor) está en un único objeto `run` dentro de `Campaign.ts`. Los accesorios modifican los multiplicadores de `run.stats`, que el jugador y la fotografía consultan.

5.3 Arte generado por código

Cada textura se describe como una cadena SVG. `art/index.ts` la convierte en imagen y la registra en Phaser. Los personajes salen de un único generador (`art/character.ts`): un *look* (piel, pelo, ropa, accesorios) más un conjunto de poses produce un spritesheet de 15 cuadros. Fondos, utilería e interfaz se generan una vez en el arranque; los personajes, por evento.

5.4 Audio

`Music.ts` es un secuenciador por pasos: cada pista define compás, acordes, batería y voces con su instrumento sintetizado (órgano, cajita de música, bajo 808, cuerdas, guitarra distorsionada, etc.). `Sfx.ts` genera los efectos y la voz de los diálogos. Ambos comparten un `AudioContext` que se habilita con la primera interacción del usuario.

6. Información técnica

6.1 Configuración de los eventos

#	Evento	Fotos mínimas	Tarjeta	Tiempo	Sensor	Maneja los láseres
1	Noche de alabanza	4	10	2:30	3	Iluminador
2	Cumpleaños infantil	4	10	2:25	3	Animador
3	Cumpleaños de 15	5	10	2:20	3	DJ
4	Boda	5	10	2:15	3	DJ
5	Discoteca	6	10	2:10	3	DJ
6	Recital	6	10	2:05	3	Técnico de luces
7	Contra el DJ	14 puntos de vida	30	3:00	5	DJ (jefe)

Cada evento es un `ScenarioConfig` (`src/config/scenarios/types.ts`) con: fondo y música, utilería y zonas sólidas, invitados (vestuario, asientos, bailarines, quienes deambulan), actores con nombre, momentos, objetivos, láseres (emisores, zona, corredor seguro y tramos de dificultad) y, opcionalmente, un jefe.

6.2 Entradas

Acción	Teclado	Táctil	Mando	Control remoto
Moverse	WASD o flechas	Palanca	Palanca o cruceta	Flechas
Sacar foto / aceptar	Espacio o Enter	Botón de cámara / toque	A	Enter
Interactuar (salir, usar objeto)	E o 0	Botón E	B o X	0
Pausa	Esc	Botón de pausa	Start	—
Efectos / música	M / N	Botones en la pausa y el menú	—	—

6.3 Datos guardados

Solo las preferencias, en `localStorage` bajo la clave `flashed.settings`: idioma, efectos y música. El progreso de la campaña no se guarda: vive en memoria y se reinicia al recargar.

6.4 Herramientas de prueba

Recurso	Uso
`?scene=Game&level=N`	Abre directamente el evento N. También `?scene=Prologue`, `Ending`, `Intro`, `Upgrade`.
`?touch`	Fuerza el modo táctil en una computadora (el mouse hace de dedo).
`?canvas`	Usa Canvas 2D en lugar de WebGL.
`__flashed`, `__flashedRun()`	Desde la consola: la instancia de Phaser y el estado de la partida.

6.5 Estructura de carpetas

Ruta	Contenido
`index.html`	Página, estilos de pantalla completa y aviso de girar el dispositivo.
`src/main.ts`	Configuración de Phaser y registro de escenas.

`src/scenes/`	Una clase por pantalla; `Cutscenes.ts` contiene las dos cinemáticas.
`src/game/`	Reglas: jugador, invitados, fotografía, láseres, HUD, colisiones, controles táctiles, campaña, accesorios.
`src/config/`	Constantes de balance y un archivo por escenario.
`src/art/`	Generadores SVG: personajes, vestuarios, escenarios, menú, cinemáticas e íconos.
`src/audio/`	Música, efectos y contexto de audio.
`src/i18n/`, `settings.ts`	Textos en dos idiomas y preferencias.
`src/device.ts`, `gamepad.ts`	Detección táctil, pantalla completa, reajuste y mando.

7. Estado de verificación y límites conocidos

Verificado

- Verificación de tipos y compilación de producción sin errores.
- Recorridos automáticos en un navegador sin interfaz: menú, selección, cinemáticas, los siete eventos, reglas de objetivos, tarjeta, láseres, jefe, accesorios y pantallas de resultado.
- Pantallas emuladas de 667 × 375 a 3840 × 2160, con y sin entrada táctil, y mando simulado.
- Dos teléfonos reales, probados por el autor del proyecto.

Pendiente o limitado

- Televisores, tabletas y mandos reales todavía no se probaron.
- El balance de dificultad con las reglas actuales (10 disparos, objetivos obligatorios) no se jugó completo a mano.
- El repositorio no incluye pruebas automáticas; los recorridos de verificación se hicieron con scripts externos.
- En 4K el juego se amplía desde 1080p. En iPhone el navegador no permite pantalla completa automática.
- Con control remoto no hay pausa. El progreso no se guarda y todos los eventos están disponibles desde el inicio.

> **Para extender el juego.** Un evento nuevo es un archivo en `src/config/scenarios/` agregado a `CAMPAIGN`, más su fondo en `src/art/`, sus textos en los dos diccionarios y, si hace falta, una pista en `Music.ts`. Un accesorio nuevo es una entrada en `UPGRADES`, un ícono y dos textos.